

Kịch bản thuyết minh (tiếng Việt) — Demo Video HW05

Đọc theo giọng của bạn, có thể chỉnh lại tự nhiên hơn. Phần trong [...] là chỗ cần nhìn màn hình để đọc số liệu thực tế lúc quay, không phải nói y nguyên.

1. Mở đầu (nói 1 lần, trước khi chạy Load)

"Em chào thầy, em là Lê Thiên Phú — mã số sinh viên 23127244. Đây là video demo cho bài tập HW05, kiểm thử hiệu năng hệ thống EShop bằng JMeter, chạy trên máy cá nhân của em — thông tin phần cứng khớp với báo cáo hardware report em đã nộp kèm.

Em sẽ chạy lần lượt 3 kịch bản: Load test trên endpoint xem chi tiết sản phẩm, Stress test trên endpoint quên mật khẩu, và Spike test trên endpoint áp mã giảm giá. Màn hình bên trái là công cụ giám sát tài nguyên **top**, theo dõi tiến trình backend Node.js; màn hình bên phải là JMeter chạy ở chế độ non-GUI."

2. Load test — read-heavy, **GET /api/products/:id**

Trước khi chạy:

"Kịch bản đầu tiên là Load test, nhắm vào endpoint xem chi tiết sản phẩm — đây là endpoint đọc dữ liệu, được gọi thường xuyên nhất trong một trang thương mại điện tử thực tế. Em cấu hình 30 luồng người dùng ảo, tăng tải dần trong 30 giây, rồi giữ tải ổn định trong 5 phút. Thời gian nghỉ giữa các request theo phân phối Gaussian, trung bình 2 giây, mô phỏng hành vi người dùng đọc thông tin sản phẩm trước khi bấm tiếp."

Trong lúc chạy (nhìn dòng **summary** + trên JMeter, đọc số liệu thực tế):

"Hiện tại throughput đang ở khoảng [X] request/giây, thời gian phản hồi trung bình [Y] mili-giây, tỷ lệ lỗi [Z]%. Bên **top**, tiến trình Node.js đang dùng khoảng [A]% CPU."

Sau khi chạy xong:

"Load test hoàn tất, tổng cộng [N] request, tỷ lệ lỗi [Z]%. Kết quả cho thấy endpoint này xử lý tải bình thường rất ổn định, đúng như kỳ vọng vì đây chỉ là một truy vấn đọc đơn giản."

3. Stress test — auth-heavy, **POST /api/forgot-password**

Trước khi chạy:

"Kịch bản thứ hai là Stress test, nhắm vào endpoint quên mật khẩu. Endpoint này không giống endpoint trước — mỗi request đều ghi dữ liệu xuống database, cụ thể là cập nhật cột reset token, nên đây là điểm dễ nghẽn hơn. Em đã thử nghiệm trước bằng cách gửi đồng thời tới 700 request cùng lúc

và phát hiện hệ thống không trả lỗi, nhưng thời gian phản hồi tăng gần như tuyến tính theo số lượng request đồng thời — vì SQLite chỉ cho phép một thao tác ghi tại một thời điểm.

Vì vậy em thiết kế bài test theo kiểu bậc thang, tăng dần từ 30, lên 80, 150, rồi 400 luồng người dùng ảo qua 4 giai đoạn liên tiếp, để tìm điểm giới hạn thực sự. Em cũng thêm một assertion kiểm tra thời gian phản hồi phải dưới 2 giây, vì chỉ kiểm tra mã phản hồi 200 sẽ không phát hiện được vấn đề nghẽn ghi dữ liệu này."

Trong lúc chạy:

"Đang ở giai đoạn [số giai đoạn], với [số luồng] luồng người dùng ảo. Thời gian phản hồi trung bình hiện tại là [Y] mili-giây — có thể thấy nó [tăng / không đổi] so với giai đoạn trước. Bên **top**, CPU của tiến trình Node.js đang ở mức [A]%."

Sau khi chạy xong:

"Stress test hoàn tất. [Nếu có lỗi/assertion fail ở giai đoạn cao]: có thể thấy ở giai đoạn 400 luồng, thời gian phản hồi đã vượt ngưỡng 2 giây, đúng như dự đoán ban đầu, chứng minh giới hạn ghi dữ liệu của SQLite. [Nếu không]: hệ thống vẫn giữ được thời gian phản hồi dưới ngưỡng ở mọi giai đoạn, dù có tăng dần."

4. Spike test — transactional, **POST /api/apply-coupon**

Trước khi chạy:

"Kịch bản cuối cùng là Spike test, nhắm vào endpoint áp mã giảm giá. Endpoint này chỉ đọc dữ liệu, không ghi, nên em đã thử nghiệm trước với burst test lên tới 2000 request đồng thời và không phát hiện lỗi nào — khác hẳn với endpoint quên mật khẩu ở bài Stress."

Vì vậy, thay vì cố tìm điểm giới hạn, bài Spike test này được thiết kế theo 3 giai đoạn: tải nền bình thường với 15 luồng, đột ngột tăng vọt lên 500 luồng chỉ trong 2 giây, rồi giảm về lại tải nền để kiểm tra khả năng phục hồi của hệ thống sau cú sốc tải."

Trong lúc chạy (đặc biệt chú ý lúc chuyển sang giai đoạn Spike):

"Đây là thời điểm tải tăng đột ngột — có thể thấy CPU bên **top** [tăng mạnh / phản ứng ra sao]. JMeter đang báo throughput [X] request/giây, tỷ lệ lỗi [Z]%."

Sau khi chạy xong:

"Spike test hoàn tất. Hệ thống không có lỗi nào trong suốt cả 3 giai đoạn, kể cả lúc tải tăng đột ngột 500 luồng — cho thấy endpoint này có khả năng chịu đựng tốt với các cú sốc tải tức thời, khác với endpoint ghi dữ liệu ở bài Stress."

5. Kết thúc (nói 1 lần, sau khi chạy xong cả 3)

"Như vậy em đã chạy xong cả 3 kịch bản Load, Stress, và Spike. Trong quá trình thiết kế test, em cũng phát hiện một vài lỗi thực tế của hệ thống: endpoint xem chi tiết sản phẩm trả kiểu dữ liệu giá không nhất quán giữa id chẵn và lẻ, trả về mã 200 thay vì 404 khi sản phẩm không tồn tại, và endpoint áp mã

giảm giá tính sai công thức giảm giá phần trăm, ra kết quả âm. Những lỗi này em sẽ ghi nhận vào GitHub Issues kèm ảnh chụp màn hình.

Cảm ơn thầy cô đã xem video demo của em."

Ghi chú khi quay

- Đọc số liệu thật từ dòng **summary** + của JMeter và từ **top** lúc quay — đừng học thuộc số, vì số liệu sẽ khác mỗi lần chạy.
- Nếu Stress test không thực sự vượt ngưỡng 2 giây khi chạy thật, cứ nói đúng sự thật ("hệ thống vẫn ổn định") — đừng nói sai kết quả để khớp kịch bản.
- Tổng thời lượng 3 đoạn ước tính: Load ~5.5 phút + Stress ~3 phút + Spike ~1 phút = đã vượt yêu cầu tối thiểu 6 phút, không cần nói dài dòng để "câu giờ".